

| 하남시 정신건강 산학협력 프로젝트

Hanam MC

하남시 정신건강복지센터 의뢰 · MENINBLOX 주관 · 서강대 가상융합대학원 수행의 산학협력 마음건강 교육 앱. 서강대 측 1인 개발로 Unity 클라이언트 전 영역(인증·DB·문제풀이·STT·인벤토리)을 단독 구축. 1차 빌드 후 2026.02 새 기획자 합류로 전면 재작업, 2026.03까지 진행하다 학교 사정으로 중단된 미완성 프로젝트지만 여기까지의 코드 베이스는 모두 본인 손으로 만든 결과물.



1인 개발

서강대 측 단독 클라이언트 담당



Unity 6

6000.0.42f1 + URP 17



92+ Scripts

4계층 아키텍처



산학협력

서강대 + MENINBLOX + 하남시



프로젝트 보기



GitHub

PROJECT OVERVIEW

게임 개요

하남시 정신건강복지센터 의뢰 · MENINBLOX 주관 · 서강대 가상융합대학원 수행의 산학협력 마음건강 교육 앱. 서강대 측 1인 개발로 Unity 클라이언트 전 영역(인증·DB·문제풀이·STT·인벤토리)을 단독 구축. 1차 빌드 후 2026.02 새 기획자 합류로 전면 재작업, 2026.03까지 진행하다

학교 사정으로 중단된 미완성 프로젝트지만 여기까지의 코드 베이스는 모두 본인 손으로 만든 결과물.



핵심 게임플레이

- ✓ 4계층 아키텍처 (UI → Service → Repository → DB) 분리 설계
- ✓ LiteDB 기반 7개 컬렉션 + 역할 기반 인증 (USER/ADMIN/SUPERADMIN)
- ✓ Whisper.cpp 임베디드 STT + 키워드 유사도 매칭
- ✓ Template Method 기반 ProblemStepBase 계층 (객관식/카드/드래그앤드롭)
- ✓ DialogueSequencer + TTS 다국어 로컬라이즈 시스템
- ✓ BCrypt(WorkFactor=10) 비밀번호 해싱 + Session 복원



개발 환경

- ✓ 엔진: Unity 6 (6000.0.42f1)
- ✓ 기간: 1차 2025.09~12 → 2026.02~03 대수정 → 학교 사정으로 중단
- ✓ 플랫폼: Windows



개발 성과

- ✓ 산학협력 프로젝트에서 서강대 측 1인으로 Unity 클라이언트 전체(92+ 스크립트) 단독 개발
- ✓ 4계층 + DI 구조로 Repository 구현체만 교체하면 서버 연동 전환 가능한 설계
- ✓ Whisper.cpp 임베디드 STT를 Unity FFI로 통합, 오프라인 음성 인터랙션 구현

TECHNOLOGY STACK

기술 스택

하남시 정신건강복지센터 의뢰 · MENINBLOX 주관 · 서강대 가상융합대학원 수행의 산학협력
마음건강 교육 앱

게임 엔진

U6

Unity 6.0.42f1

Windows 단일 타겟

URP

URP 17.0.4

Universal RP

TMP

TextMeshPro

UI 텍스트

IS

Input System 1.13

신 입력 시스템



데이터

LDB

LiteDB

임베디드 NoSQL



Repository

DB 추상화



음성



Whisper.cpp

오프라인 STT



KeywordMatcher

유사도 매칭



보안



BCrypt

WorkFactor=10



Session

역할 기반 라우팅

CORE FEATURES

핵심 기능 구현

6가지 주요 시스템과 기술적 구현 상세

```
// Hanam_MC - 하남시 보건소 산학협력 프로젝트
// - Graduate School of Virtual Convergence, Sogang University
// - MENINBLOX (주관 기업)
// - Hanam Mental Health Welfare Center (의뢰)
//
// 서강대 측 1인 개발 / Unity 클라이언트 전 영역 담당
//
// Timeline
// 2025.09 ~ 2025.12 1차 빌드 완료
// 2026.01 일시 중단
// 2026.02 ~ 2026.03 새 기획자 합류, 전면 재작업
// 학교 사정으로 중단 (미완성)
//
// Tech
// Unity 6 (6000.0.42f1) + URP 17.0.4
// C# / LiteDB / Whisper.cpp (FFI) / BCrypt.Net
// Platform: Windows
```

1. 프로젝트 배경 / 산학협력 컨텍스트

→ 협력 구조

- ****의뢰****: 하남시 정신건강복지센터
- ****주관 기업****: MENINBLOX
- ****수행 기관****: 서강대학교 가상융합대학원

→ 본인의 위치

- 서강대 측에서 ****혼자 Unity 클라이언트 전체를 담당****
- 92개 이상의 C# 스크립트, DB·인증·문제풀이·STT·인벤토리 전 영역 구현

→ 일정 흐름

- 2025.09 ~ 2025.12: 1차 빌드 완료 (인증·문제 시스템·관리자 화면)
- 2026.01: 일시 중단
- 2026.02: MENINBLOX 측 새 기획자 합류 → 기획 전면 변경에 따라 ****클라이언트 전체 재설계·재구현****
- 2026.03: 대수정 작업 진행
- ****학교 사정으로 프로젝트 중단**** → 미완성 상태로 종료

→ 포트폴리오 의미

- 완성품은 아니지만 ****여기까지의 코드 베이스는 전적으로 본인 1인의 결과물****
- 외부 기획 변경(2026.02 대수정)을 건디는 추상화·계층 분리 설계의 가치를 검증한 사례

- 정신건강 교육 콘텐츠 특성상 사용자 친화 UX, 단계별 진행도 추적, 음성 인터랙션이 핵심 요구사항으로 도출됨

```

public class DataService : MonoBehaviour
{
    public static DataService Instance { get; private set; }

    public IAuthService Auth { get; private set; }
    public IProgressService Progress { get; private set; }
    public IRewardService Reward { get; private set; }
    public IAdminDataService Admin { get; private set; }

    void Awake()
    {
        if (Instance != null) { Destroy(gameObject); return; }
        Instance = this;
        DontDestroyOnLoad(gameObject);

        // 1. Infrastructure
        Db = new DBGateway();
        var dbCore = (IDBGateway)Db;

        // 2. Repository (DBGateway 주입)
        UserRepository = new UserRepository(dbCore);
        ProgressRepository = new ProgressRepository(dbCore);
        InventoryRepository = new InventoryRepository(dbCore);
        ResultRepository = new ResultRepository(dbCore);

        // 3. Service (Repository 주입, Service 간 조합 허용)
        Auth = new AuthService(UserRepository);
        Progress = new LocalProgressService(
            ProgressRepository, UserRepository, ResultRepository);
        Reward = new LocalRewardService(
            InventoryRepository, UserRepository, Progress);
    }
}

```

2. 4계층 아키텍처 / 의존성 주입

→ 레이어 분리

- ****Presentation****: LoginFormUI, HomeSceneUI, ProblemStepBase
- ****Application****: AuthService, ProgressService, RewardService, AdminService
- ****Data Access****: UserRepository, ProgressRepository, InventoryRepository 등
- ****Infrastructure****: DBGateway → DBHelper → LiteDB

→ 설계 원칙

- 상위 레이어가 하위 인터페이스(IUserRepository 등)에만 의존하도록 DIP 적용
- DataService가 Singleton Hub 역할로 모든 Service/Repository를 생성·조립
- Service ↔ Service 의존도 허용 (LocalRewardService → IProgressService)

→ 의도

- 추후 서버 연동 전환 시 Repository 구현체만 교체하면 되도록 설계


```

public abstract class ProblemStepBase : MonoBehaviour
{
    [SerializeField] protected ProblemContext context;
    [SerializeField] protected StepKeyConfig stepKeyConfig;

    public abstract void OnStepEnter();
    public virtual void OnStepExit() { }

    protected void SaveAttempt(object body)
    {
        var key = stepKeyConfig.BuildKey(
            ProblemSession.CurrentTheme,
            ProblemSession.CurrentProblemIndex);

        DataService.Instance.Progress
            .SaveStepAttemptForCurrentUser(key, body);
    }

    protected void SaveReward(object body, string itemId, string itemName)
    {
        DataService.Instance.Reward
            .SaveRewardForCurrentUser(
                ProblemSession.CurrentTheme,
                ProblemSession.CurrentProblemIndex,
                body, itemId, itemName);
    }
}

public abstract class MultipleChoiceStepBase<T> : ProblemStepBase
{
    [SerializeField] protected StepCompletionGate completionGate;

    public abstract int QuestionCount { get; }
    public abstract T GetQuestion(int index);
    public abstract int GetCorrectOptionIndex(T q);
}

```

3. ProblemStepBase / Template Method

→ Step 계층

- ProblemStepBase (abstract)
- MultipleChoiceStepBase<T> : 객관식
- RandomCardSequenceStepBase : 카드 순서 맞추기
- InventoryDropTargetStepBase : 드래그 앤 드롭
- CommonRewardStep : 공통 보상 연출

→ 공통 혹

- OnStepEnter / OnStepExit

- SaveAttempt(body) → DB attempts 컬렉션 INSERT
- SaveReward(body, itemId, itemName) → attempts + inventory 동시 처리

→ 흐름

- StepFlowController가 List<GameObject> stepPanels 순차 활성화
- StepCompletionGate가 진행률을 카운트하고 완료 시 자동 NextStep 호출
- Director_Problem{N}_Step{M}_Logic(abstract) ↔ Binder(SerializeField 주입) 분리

```

public class MicRecordingIndicator : MonoBehaviour
{
    [SerializeField] Image targetImage;
    [SerializeField] Sprite idleSprite;
    [SerializeField] Sprite recordingSprite;
    [SerializeField] Sprite recognizingSprite;
    [SerializeField] string[] keywords;

    public event Action<int> OnKeywordMatched;
    public event Action<string> OnNoMatch;

    void OnEnable()
    {
        STTManager.Instance.OnPartialResult += HandlePartial;
        STTManager.Instance.OnFinalResult += HandleFinal;
    }

    public void ToggleRecording()
    {
        if (STTManager.Instance.IsRecording)
        {
            STTManager.Instance.StopRecording(skipProcessing: false);
            SetState(recognizingSprite);
        }
        else
        {
            STTManager.Instance.SetPromptHint(keywords);
            STTManager.Instance.StartRecording();
            SetState(recordingSprite);
        }
    }

    void HandleFinal(string text)
    {
        var (bestIdx, score) = KeywordMatcher.FindBestMatch(text, keywords);
        if (score >= 0.6f) OnKeywordMatched?.Invoke(bestIdx);
        else OnNoMatch?.Invoke(text);
        SetState(idleSprite);
    }
}

```

4. Whisper STT + 키워드 매칭

→ 오프라인 음성 인식

- Whisper.cpp 라이브러리를 FFI(IntPtr)로 직접 호출
- 마이크 → AudioClip → float 샘플 → Whisper 컨텍스트로 전달
- 무음 감지 기반 자동 종료 (사용자가 말 끝났을 때 자동 stop)

→ 3상태 시각 피드백 (MicRecordingIndicator)

- idle / recording / recognizing 스프라이트 전환
- 부분 인식(OnPartialResult)과 최종 결과(OnFinalResult) 모두 키워드 매칭에 사용

→ KeywordMatcher

- Levenshtein 기반 유사도 계산
- 부분 일치 / 전체 일치 모두 허용 → 발화 변형("마음렌즈", "마음 렌즈")에 강건
- 정신건강 콘텐츠 특성상 음성 답변을 통한 표현·감정 학습이 핵심 인터랙션

```

public class DialogueSequencer : MonoBehaviour
{
    [SerializeField] int[] enterTextIds;
    [SerializeField] int[] completedTextIds;
    [SerializeField] Text dialogueText;
    [SerializeField] Button nextDialogueBtn;
    [SerializeField] Button nextStepBtn;

    public event Action OnEnterSequenceDone;
    public event Action OnEnterComplete;

    int _index;
    int[] _current;

    public void PlaySequence(int[] ids, Action onSeq, Action onComplete)
    {
        _current = ids;
        _index = 0;
        OnEnterSequenceDone = onSeq;
        OnEnterComplete = onComplete;
        ShowAt(_index);
    }

    void ShowAt(int i)
    {
        int textId = _current[i];
        dialogueText.text = ProblemRuntime.L(textId);
        SoundManager.Instance.PlayTTS(textId);
    }

    public void OnNextDialogueClicked()
    {
        if (SoundManager.Instance.IsTTSPlaying) return;

        _index++;
        if (_index >= _current.Length) OnEnterSequenceDone?.Invoke();
        else ShowAt(_index);
    }
}

```

5. DialogueSequencer / TTS 로컬라이즈

→ 다국어 텍스트 → 음성 파이프라인

- ProblemRuntime.L(textId) / LK / LE 로 한·영·기타 언어 텍스트 조회
- SoundManager가 textId → AudioClip 매핑 사전을 보유, 동일 ID로 TTS 재생
- 텍스트와 음성이 같은 식별자로 묶여 있어 누락·미스매치 방지

→ 시퀀스 흐름

- enterTextIds[] : Step 진입 시 자동 재생할 대사 배열
- completedTextIds[] : 게이트 완료 시 재생할 마무리 대사

- 다음 대사 버튼 ↔ 다음 단계 버튼이 이벤트로 분리되어 사용자 흐름 제어

```
public class AuthService : IAuthService
{
    private readonly IUserRepository _users;
    private const int BcryptWorkFactor = 10;

    public AuthService(IUserRepository users)
    {
        _users = users ?? throw new ArgumentNullException(nameof(users));
        EnsureSuperAdmin();
    }

    public Result<User> Login(string email, string password)
    {
        try
        {
            var e = AuthValidator.NormalizeEmail(email);
            if (!AuthValidator.IsValidEmail(e))
                return Result<User>.Fail(AuthError.EmailInvalid);

            var u = _users.FindActiveUserByEmail(e);
            if (u == null)
                return Result<User>.Fail(AuthError.NotFoundOrInactive);

            bool ok = BCrypt.Net.BCrypt.Verify(password, u.PasswordHash);
            if (!ok)
                return Result<User>.Fail(AuthError.PasswordMismatch);

            return Result<User>.Success(u);
        }
        catch (Exception ex)
        {
            Debug.LogError($"[AuthService] Login error: {ex}");
            return Result<User>.Fail(AuthError.Internal);
        }
    }

    public Result SignUp(string name, string email, string password)
    {
        if (_users.ExistsEmail(email))
            return Result.Fail(AuthError.EmailDuplicated);

        var hash = BCrypt.Net.BCrypt.HashPassword(password, BcryptWorkFactor);
        _users.InsertUser(new User { Name = name, Email = email, PasswordHash = hash });
        return Result.Success();
    }
}
```

6. 인증 / 보안 (BCrypt + Result 모노이드)

→ BCrypt 해싱

- WorkFactor = 10 으로 회원가입 시 PasswordHash 저장
- 로그인 시 Verify로만 비교, 평문 비밀번호는 절대 보관/로그하지 않음

→ Result 모노이드

- Result<T> / AuthError enum 으로 명시적 에러 흐름
- try-catch 후 Internal 에러로 분기, UI는 enum만 보고 메시지 결정

→ 역할 기반 라우팅

- 로그인 성공 시 SessionManager.SignIn → 역할 따라 분기
- USER → HomeScene (문제 풀이)
- ADMIN/SUPERADMIN → ResultScene (관리자 화면)

→ SuperAdmin Bootstrap

- AuthService 생성자에서 EnsureSuperAdmin() 호출, 빈 DB일 때 기본 관리자 자동 생성

DESIGN PATTERNS

디자인 패턴

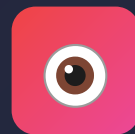
확장 가능하고 유지보수하기 쉬운 코드 아키텍처



Singleton

DataService, SessionManager,
SoundManager, STTManager,
ProblemRuntime 전역 허브

✓ 적용 완료



Repository

User/Progress/Inventory/Problem/Result/Feedback
Repository로 LiteDB 추상화

✓ 적용 완료



Service Layer

AuthService, ProgressService, RewardService
비즈니스 로직 캡슐화

✓ 적용 완료

Template Method

ProblemStepBase의 OnStepEnter/Exit 혹, 하
위 클래스에서 세부 구현

✓ 적용 완료



Binder/Logic

Director_Problem{N}_Step{M}_Logic(abstract)
↔ Binder(SerializeField) 분리

✓ 적용 완료



Observer

DialogueSequencer, MicRecordingIndicator,
StepCompletionGate의 C# event 통신

✓ 적용 완료



State

MicRecordingIndicator의
idle/recording/recognizing 3상태 시각 피드
백

✓ 적용 완료



Facade

DataService가 Auth/Progress/Reward 등 하
위 서비스 통합 진입점 제공

✓ 적용 완료

ACHIEVEMENTS

주요 성과

프로젝트를 통해 달성한 기술적 성과와 학습 내용



기술적 성과

- ★ 산학협력 프로젝트에서 서강대 측 1인으로 Unity 클라이언트 전체(92+ 스크립트) 단독 개발
- ★ 4계층 + DI 구조로 Repository 구현체만 교체하면 서버 연동 전환 가능한 설계
- ★ Whisper.cpp 임베디드 STT를 Unity FFI로 통합, 오프라인 음성 인터랙션 구현
- ★ BCrypt + Result 모노이드 + 역할 기반 라우팅으로 보안·에러 흐름 명시화



배운점

- ★ 외부 기획 전면 변경(2026.02 대수정)을 클라이언트가 견디기 위한 추상화·계층 분리의 실질적 가치 체감
- ★ Singleton 허브(DataService)를 통한 의존성 조립이 1인 개발에서 변경 비용을 크게 낮춤
- ★ Template Method + Binder/Logic 분리로 다수 Step UI 변형을 안전하게 확장
- ★ FFI 기반 외부 라이브러리(Whisper.cpp) 통합 시 메모리·스레드 수명 관리의 중요성
- ★ 프로젝트 중단이라는 외부 변수에도 코드 자체는 자산으로 남는다는 것을 인지하고, 이후 개인 작업의 문서화·구조 설계 우선순위를 재정립

SCREENSHOTS & VIDEOS

게임 미디어

게임 플레이 스크린샷과 영상

 스크린샷 1

 스크린샷 2

 스크린샷 3



Hanam MC

로 개발한 하남시 정신건강 산학협력 프로젝트 포트폴리오

© All Rights Reserved

프로젝트 링크



GitHub Repository



홈으로 돌아가기

기술 스택

Unity 6

C#

URP

LiteDB

Whisper STT

BCrypt